

■ Sockets Programming Cheat Sheet

Client–Server Communication in C · Beginner-Friendly Reference

■ WHAT IS A SOCKET?

A **socket** is an endpoint for network communication — think of it as a special file that lets two programs talk over a network. When you open a socket in C, the OS gives you a **file descriptor** (just an integer). You then use **send()** and **recv()** on that descriptor exactly as you would use **write()** / **read()** on a regular file.

Socket Call	Telephone Analogy	What it does
<code>socket()</code>	Buy a telephone	Create a new socket endpoint
<code>bind()</code>	Plug in / Insert a SIM	Assign a port number to the socket
<code>listen()</code>	Wait for ringtone	Mark socket as passive (server only)
<code>accept()</code>	Pick up the phone	Accept an incoming connection
<code>connect()</code>	Dial a number	Client initiates connection to server
<code>send()</code> / <code>recv()</code>	Listen and talk	Exchange data over the connection
<code>close()</code>	Hang up	Terminate the connection

■ CLIENT vs SERVER — BIG PICTURE

SERVER	CLIENT
Waits for connections	Initiates the connection
Binds to a well-known port (e.g. 3490)	OS picks an ephemeral port automatically
Calls: <code>socket</code> → <code>bind</code> → <code>listen</code> → <code>accept</code>	Calls: <code>socket</code> → <code>connect</code>
<code>accept()</code> returns a NEW socket for each client	<code>connect()</code> uses the single socket directly
Can serve many clients simultaneously (fork/threads)	Connects to one server at a time

■ KEY VOCABULARY

Port number — A 16-bit number (0–65535) that identifies a specific service on a host. Well-known ports: 80=HTTP, 443=HTTPS, 22=SSH. Your programs usually pick something >1024.

IP address — A numerical label (e.g. 192.168.1.1 for IPv4) that identifies a machine on a network.

TCP — Transmission Control Protocol — reliable, ordered, connection-based. Use `SOCK_STREAM`.

UDP — User Datagram Protocol — fast, connectionless, no guarantee of delivery. Use `SOCK_DGRAM`.

File descriptor (fd) — An integer the OS gives you to refer to an open resource (file, socket, pipe).

Ephemeral port — A temporary high-numbered port the OS assigns automatically to a client or accepted connection.

sockaddr / sockaddr_in — C structs that hold address information (family, IP, port). `sockaddr` is the generic; `sockaddr_in` is the IPv4 version.

addrinfo — A newer struct that makes it easier to set up addresses — works for both IPv4 and IPv6.

getaddrinfo() — A function that resolves a hostname + port into a linked list of ready-to-use `addrinfo` structs.

Blocking call — A function that does not return until its job is done (e.g. `accept()` waits until a client connects).

■■ SERVER — STEP-BY-STEP WALKTHROUGH

STEP 1 — Set up hints and call `getaddrinfo()`

getaddrinfo() converts a hostname + port string into a linked list of **struct addrinfo** items that you can pass directly to **socket()** and **bind()**. On the server side, pass **NULL** for the hostname and set the **AI_PASSIVE** flag — this means "use my own IP".

```
■ server_setup.c – getaddrinfo
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>

struct addrinfo hints, *servinfo;
memset(&hints, 0, sizeof hints); // zero out the struct first!
hints.ai_family = AF_UNSPEC; // IPv4 or IPv6 – we don't care
hints.ai_socktype = SOCK_STREAM; // TCP (use SOCK_DGRAM for UDP)
hints.ai_flags = AI_PASSIVE; // fill in my IP automatically
if (getaddrinfo(NULL, "3490", &hints, &servinfo) != 0)
    die("getaddrinfo failed");
// servinfo is now a linked list of address options
```

STEP 2 — Create socket, set options, bind

Loop over the linked list until you find an address that works. **setsockopt(SO_REUSEADDR)** lets you restart the server quickly without waiting for the OS to release the port.

```
■ server_setup.c – socket/bind
int sockfd, yes = 1;
struct addrinfo *p;
for (p = servinfo; p != NULL; p = p->ai_next) {
    // 1. Create the socket
    sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (sockfd == -1) continue; // try next address
    // 2. Allow reuse of port (avoids 'Address already in use' errors)
    setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(int));
    // 3. Bind the socket to the port
    if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
        close(sockfd);
        continue; // try next
    }
    break; // success – stop looping
}
freeaddrinfo(servinfo); // always free the list when done
```

STEP 3 — listen()

Puts the socket into passive mode. The second argument is the **backlog** — how many pending connections the OS will queue up before refusing new ones. 10 is a typical value.

```
■ server_setup.c – listen
// Allow up to 10 pending connections in the queue
if (listen(sockfd, 10) == -1)
    die("listen failed");
```

STEP 4 — accept() in a loop

accept() blocks until a client connects. When it returns, it gives you a **brand-new socket file descriptor** (chatSocket) dedicated to that one client. The original **sockfd** stays open to accept more clients.

```
■ server_setup.c – accept loop
struct sockaddr_storage client_addr; // big enough for IPv4 or IPv6
while (1) { // server runs forever
    socklen_t clientSize = sizeof(client_addr);
    // accept() blocks here until a client connects
    int chatSocket = accept(sockfd,
        (struct sockaddr *)&client_addr,
```

```

&clientSize);
if (chatSocket == -1) { perror('accept'); continue; }
printf("Client connected! chatSocket fd = %d\n", chatSocket);
// === talk to client using chatSocket ===
char buf[256];
int bytes = recv(chatSocket, buf, sizeof(buf)-1, 0);
buf[bytes] = '\0';
send(chatSocket, "Hello!", 6, 0);
// =====
close(chatSocket); // done with THIS client
// loop back – sockfd is still listening
}

```

■■ **accept() and connect() are BLOCKING** — the program pauses there until something happens. To serve multiple clients at once, use **fork()** or **threads** (see last section).

■ CLIENT — STEP-BY-STEP WALKTHROUGH

STEP 1 — **getaddrinfo()** for the server's address

Same function as the server, but now you pass the **actual server hostname** (or IP address) as the first argument. No **AI_PASSIVE** flag — you're connecting outward, not listening.

```

■ client.c – getaddrinfo
struct addrinfo hints, *servinfo;
memset(&hints, 0, sizeof hints);
hints.ai_family = AF_UNSPEC; // IPv4 or IPv6
hints.ai_socktype = SOCK_STREAM; // TCP
// NO ai_flags = AI_PASSIVE here – client doesn't need it
// First arg is the server hostname or IP, second is port
if (getaddrinfo("www.uconn.edu", "3490", &hints, &servinfo) != 0)
die("getaddrinfo failed");

```

STEP 2 — **socket()** then **connect()**

No **bind()** needed — the OS automatically picks a free ephemeral port for the client. **connect()** blocks until the server accepts or refuses.

```

■ client.c – connect
int sockfd;
struct addrinfo *p;
for (p = servinfo; p != NULL; p = p->ai_next) {
sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
if (sockfd == -1) continue;
// connect() blocks until server accepts (or fails)
if (connect(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
close(sockfd);
continue;
}
break; // connected!
}
freeaddrinfo(servinfo);

```

STEP 3 — **send() / recv()** then **close()**

```

■ client.c – send/recv/close
// Send a message
char *msg = "Hello Server!";
int bytes_sent = send(sockfd, msg, strlen(msg), 0);

```

```
// Receive a reply – put in a loop for large data!
char buf[512];
int bytes_recv = recv(sockfd, buf, sizeof(buf)-1, 0);
buf[bytes_recv] = '\0';
printf("Server replied: %s\n", buf);

// Done
close(sockfd);
```

■ Always put `send()` and `recv()` in a loop! A single call may not send/receive all the bytes at once — especially for large messages.

■ DATA STRUCTURES EXPLAINED

struct addrinfo (the modern way)

This struct packages everything `socket()` and `bind()/connect()` need. `getaddrinfo()` fills it in for you — you rarely need to set fields manually.

```
■ netdb.h – struct addrinfo
struct addrinfo {
int ai_flags; // AI_PASSIVE (server) or 0 (client)
int ai_family; // AF_INET (IPv4), AF_INET6 (IPv6), AF_UNSPEC (either)
int ai_socktype; // SOCK_STREAM (TCP) or SOCK_DGRAM (UDP)
int ai_protocol; // usually 0 = auto
size_t ai_addrlen; // size of ai_addr in bytes
struct sockaddr *ai_addr; // pointer to actual address (sockaddr_in or _in6)
char *ai_canonname; // full hostname
struct addrinfo *ai_next; // LINKED LIST – always loop through all!
};
```

struct sockaddr (generic) vs struct sockaddr_in (IPv4)

`sockaddr` is a generic placeholder. The actual data lives in `sockaddr_in` (IPv4) or `sockaddr_in6` (IPv6). You cast between them — the compiler is fine with this because the memory layout is compatible.

```
■ sys/socket.h – address structs
// Generic – used as parameter type in bind(), connect(), accept()
struct sockaddr {
unsigned short sa_family; // AF_INET or AF_INET6
char sa_data[14]; // address data (placeholder)
};

// IPv4 specific – this is what you actually fill in
struct sockaddr_in {
short sin_family; // = AF_INET
unsigned short sin_port; // port in network byte order (use htons(!))
struct in_addr sin_addr; // IP address
};

// Cast when calling bind/connect:
bind(sockfd, (struct sockaddr *)&my_addr, sizeof(my_addr));
```

■ You almost never fill in `sockaddr_in` by hand — just let `getaddrinfo()` do it and pass the result's `ai_addr` / `ai_addrlen` directly.

struct sockaddr_storage (client address in accept)

When you call `accept()`, you need a buffer to receive the connecting client's address. Use `sockaddr_storage` — it's large enough to hold any address type (IPv4 or IPv6) without you knowing in advance which it will be.

```
■ accept() pattern
struct sockaddr_storage client_addr; // big enough for anything
socklen_t clientSize = sizeof(client_addr);
int chatSocket = accept(sockfd, (struct sockaddr *)&client_addr, &clientSize);
```

■ FUNCTION SIGNATURES & RETURN VALUES

socket()

```
int socket(int domain, int type, int protocol)
```

Returns: Returns fd (≥ 0) on success, -1 on error.

Notes: domain=AF_INET or AF_UNSPEC, type=SOCK_STREAM (TCP), protocol=0 (auto)

bind()

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen)
```

Returns: Returns 0 on success, -1 on error.

Notes: Assign a local address/port to sockfd. Server only.

listen()

```
int listen(int sockfd, int backlog)
```

Returns: Returns 0 on success, -1 on error.

Notes: backlog = max pending connection queue size (e.g. 10).

accept()

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen)
```

Returns: Returns NEW socket fd (≥ 0) on success, -1 on error. BLOCKS.

Notes: sockfd = listening socket. Returns dedicated socket for THIS client.

connect()

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen)
```

Returns: Returns 0 on success, -1 on error. BLOCKS.

Notes: Client calls this. No bind() needed first.

send()

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags)
```

Returns: Returns bytes sent (may be less than len!), -1 on error.

Notes: flags=0 for normal use. Use in a loop for reliability.

recv()

```
ssize_t recv(int sockfd, void *buf, size_t len, int flags)
```

Returns: Returns bytes received (0 = connection closed), -1 on error. BLOCKS.

Notes: flags=0 for normal use. Use in a loop for large data.

close()

```
int close(int sockfd)
```

Returns: Returns 0 on success, -1 on error.

Notes: Closes the socket. Always close chatSocket after each client.

getaddrinfo()

```
int getaddrinfo(const char *node, const char *service, const struct addrinfo *hints, struct addrinfo **res)
```

Returns: Returns 0 on success, non-zero error code on failure.

Notes: node=hostname or IP (NULL for server), service=port or service name.

freeaddrinfo()

```
void freeaddrinfo(struct addrinfo *res)
```

Returns: No return value.

Notes: Always call this after getaddrinfo() to free the linked list.

■ COMPLETE CODE TEMPLATES

Minimal TCP Server (copy-paste ready)

```
■ server.c
```

```
#include <stdio.h>
```

```
#include <string.h>
```

```

#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <unistd.h>

int main(void) {
    struct addrinfo hints, *servinfo, *p;
    int sockfd, chatSocket, yes = 1;
    char buf[1024];

    memset(&hints, 0, sizeof hints);
    hints.ai_family = AF_UNSPEC;
    hints.ai_socktype = SOCK_STREAM;
    hints.ai_flags = AI_PASSIVE;

    getaddrinfo(NULL, "3490", &hints, &servinfo);
    for (p = servinfo; p != NULL; p = p->ai_next) {
        sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
        if (sockfd == -1) continue;
        setsockopt(sockfd, SOL_SOCKET, SO_REUSEADDR, &yes, sizeof(int));
        if (bind(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
            close(sockfd); continue;
        }
        break;
    }
    freeaddrinfo(servinfo);
    listen(sockfd, 10);
    printf("Server listening on port 3490...\n");
    while (1) {
        struct sockaddr_storage client_addr;
        socklen_t sz = sizeof(client_addr);
        chatSocket = accept(sockfd, (struct sockaddr *)&client_addr, &sz);
        if (chatSocket == -1) continue;

        int n = recv(chatSocket, buf, sizeof(buf)-1, 0);
        if (n > 0) {
            buf[n] = '\0';
            printf("Received: %s\n", buf);
            send(chatSocket, "ACK", 3, 0);
        }
        close(chatSocket);
    }
    return 0;
}

```

Minimal TCP Client (copy-paste ready)

```

■ client.c
#include <stdio.h>
#include <string.h>
#include <sys/types.h>
#include <sys/socket.h>
#include <netdb.h>
#include <unistd.h>

int main(void) {
    struct addrinfo hints, *servinfo, *p;
    int sockfd;
    char buf[1024];

    memset(&hints, 0, sizeof hints);

```

```

hints.ai_family = AF_UNSPEC;
hints.ai_socktype = SOCK_STREAM;
// NO AI_PASSIVE for client!

getaddrinfo("localhost", "3490", &hints, &servinfo);
for (p = servinfo; p != NULL; p = p->ai_next) {
    sockfd = socket(p->ai_family, p->ai_socktype, p->ai_protocol);
    if (sockfd == -1) continue;
    if (connect(sockfd, p->ai_addr, p->ai_addrlen) == -1) {
        close(sockfd); continue;
    }
    break;
}
freeaddrinfo(servinfo);

// Send a message
send(sockfd, "Hello!", 6, 0);

// Receive reply
int n = recv(sockfd, buf, sizeof(buf)-1, 0);
buf[n] = '\0';
printf("Server said: %s\n", buf);
close(sockfd);
return 0;
}

```

■ HANDLING MULTIPLE CLIENTS

The basic `accept()` loop handles only one client at a time. While you're talking to client A, client B is stuck waiting in the queue. Real servers solve this with **`fork()`** (processes), **threads**, or I/O multiplexing (`select()` / `poll()`).

Option 1 — `fork()` a child process per client

```

■ fork pattern
while (1) {
    chatSocket = accept(sockfd, (struct sockaddr *)&client_addr, &sz);
    if (fork() == 0) {
        // *** CHILD process – handles this one client ***
        close(sockfd); // child doesn't need listening socket
        handle_client(chatSocket);
        close(chatSocket);
        exit(0);
    }
    // *** PARENT process – immediately loops back to accept() ***
    close(chatSocket); // parent doesn't need chatSocket
}

```

■ After `fork()`, the child gets its own copy of all file descriptors. Close the ones you don't need in each process — otherwise sockets won't close properly.

Option 2 — `select()` / `poll()` (single process, I/O multiplexing)

`select()` lets one process monitor multiple file descriptors at once and react to whichever becomes ready. This is more complex but avoids the overhead of spawning processes. Your slides mention this as an alternative to `fork` — know that it exists for the exam.

■■ COMMON MISTAKES & EXAM TRAPS

■ Forgetting to loop `send()/recv()`

A single `send()` or `recv()` may transfer fewer bytes than you requested. Always loop until all bytes are sent/received.

■ Not calling freeaddrinfo()

getaddrinfo() allocates a linked list on the heap. Always call freeaddrinfo(servinfo) when done or you'll leak memory.

■ Using AI_PASSIVE on the client

AI_PASSIVE means 'use my own IP' — it's for servers only. Clients specify the server's hostname in node, no flag needed.

■ Forgetting setsockopt(SO_REUSEADDR)

Without this, restarting a server quickly gives 'Address already in use'. Always set SO_REUSEADDR before bind().

■ Closing sockfd instead of chatSocket

After accept(), close chatSocket (the per-client socket) when done. Closing sockfd would stop the server from accepting new clients.

■ Not checking return values

socket(), bind(), listen(), accept(), connect() all return -1 on failure. Always check and handle errors (perror() is your friend).

■ bind() on client side

Clients do NOT call bind(). The OS automatically assigns an ephemeral port when connect() is called.

■ Port byte order

Port numbers in sockaddr_in must be in network byte order. Use htons(port) to convert. getaddrinfo() handles this for you automatically.

■ QUICK REFERENCE CARD — SERVER vs CLIENT FLOW

Step	SERVER	CLIENT
1	getaddrinfo(NULL, port, &hints;, &res;) Set AI_PASSIVE flag	getaddrinfo(hostname, port, &hints;, &res;) No AI_PASSIVE
2	socket(family, type, proto)	socket(family, type, proto)
3	setsockopt(SO_REUSEADDR) bind(sockfd, addr, addrlen)	(skip — no bind needed)
4	listen(sockfd, backlog)	connect(sockfd, addr, addrlen) ← BLOCKS
5	chatSocket = accept(sockfd, ...) ← BLOCKS	(connected)
6	send() / recv() on chatSocket	send() / recv() on sockfd
7	close(chatSocket) (loop back to accept())	close(sockfd)

■ REQUIRED HEADERS

■ Required includes

```
#include <sys/types.h> // basic type definitions
#include <sys/socket.h> // socket(), bind(), listen(), accept(), connect()
#include <netdb.h> // getaddrinfo(), freeaddrinfo(), struct addrinfo
#include <arpa/inet.h> // inet_ntop() — convert IP to string for printing
#include <unistd.h> // close()
#include <string.h> // memset()
#include <stdio.h> // printf(), perror()
#include <stdlib.h> // exit()
```

■ KEY CONSTANTS

Constant	Value / Meaning
AF_INET	IPv4 address family
AF_INET6	IPv6 address family
AF_UNSPEC	Don't care — IPv4 or IPv6
SOCK_STREAM	TCP — reliable, ordered, connected
SOCK_DGRAM	UDP — fast, connectionless, unreliable

AI_PASSIVE	Fill in my own IP (server only)
SOL_SOCKET	Socket-level option (used with setsockopt)
SO_REUSEADDR	Allow reuse of local address/port

■ Good luck on your exam! Remember the flow: server = socket→bind→listen→accept | client = socket→connect. Both sides then send()/recv() in a loop, then close().